Student ID or IDs for Group Work	5663262

Date Set	November 2024
Submission Date (Excluding Extensions)	Monday 09th December 2024 by 12:00pm (UK Time)
Submission guidance	To be submitted electronically via Tabula.
Late Submission Policy	If work is submitted late, penalties will be applied at the rate of **5 marks per University working day** after the due date, up to a **maximum of 10 working days** late. After this period the mark for the work will be reduced to 0 (which is the maximum penalty). "Late" means **after the submission deadline time as well as the date** – work submitted after the given time even on the same day is counted as 1 day late. For **Postgraduate** students only, who started their **current course before 1 August 2019**, the daily penalty is **3 marks** rather than 5.
Resit Policy	If you fail this Module and/or Component, the University allows Students to remedy failure (within certain limits). Decisions to authorise Resits are made by Exam Boards. These will be issued at specific times of the year, depending on your Programme of Study. More information can be found from your Programme Office if you are concerned. If this is **already a RESIT** attempt, this means you will not be eligible for an additional attempt. The University allows as standard a maximum of two attempts on any Assessment (i.e. only one Resit). Students can only have a third attempt under exceptional circumstances via a Mitigating Circumstances Panel Decision.

Module Title & Code	Managing Cyber Risk, Audit and Compliance (WM9C4-15) **MAINSIT**
Module Leader	Sandy Taramonli
Module Tutor	E.B, Rajan Bindra, Pardeep Sandhu
Assessment Type	Report
Weighting of mark	80%

Assignment Brief is at the end of this Document.	
Word Count	The maximum word count is 3200 words. +10% will be allowed on this word count without penalty. If you exceed this word count by more than 10% to 30%, a penalty of 10% on the original mark awarded will be applied. If you exceed this word count by more than 30%, the final mark will be capped at the pass mark. The word count does not include tables; however, you must use tables carefully. Improper use of tables, e.g. as an opportunity to consume more words will be penalised. The word count does not include references, or appendices. Although appendices are allowed, these will not be marked and only referred to add context, please do not place into the appendices anything which you materially rely upon in your submission.
Module Learning Outcomes (numbered)	**1. Demonstrate a critical awareness of the process of auditing Information Systems.** 2. Collaboratively formulate and apply a Compliance Framework to respond to a given scenario. **3. Develop a Risk Assessment response to a given Case Study.** 4. **Report on the outcome of a detailed Risk Analysis Exercise for specified Stakeholders.**
Learning Outcomes Assessed in this Assessment (numbered)	LO1, LO3, LO4
Academic Guidance Resources	You will have an opportunity to ask questions and get support on the Assessment after it has been handed to you. You will be supported in this Assessment through the dedicated Teams Channel. **<u>Notes to Students</u>** • If support is provided on a Teams Channel or a Moodle Forum, please ensure you check previous questions posted on the Channel. The Teams/Moodle Channel will typically be closed the day before the Submission Date and no new questions will be addressed, therefore, please organize your time accordingly. • Please be patient with Module Tutors. • Please turn on your Teams Channel/Moodle notifications. If a Tutor has not responded to a query within 5 working days, please email the Module Tutor.

Table of content

1. Introduction **4**

2. Design Justification **4**

2.1 System Architecture 4

2.2 Cryptographic Method Selection and Applicability 7

2.3 Explanation 11

3. Implementation Details

3.1 Main function 12

3.1.1 Encryption/Decryption Module 12

3.1.2 Key Management Strategy 13

3.1.3 Secure Communication Module 13

3.1.4 Authentication and Access Control 14

3.1.5 Data Integrity Protection 15

3.2 Functionality and Encryption Methods 16

3.3 User Role Functionality Implementation 17

3.3.1 Client (Customer) Functionalities 18

3.3.2 Bank Employee Functionalities 19

3.3.3 System Administrator Security Management 20

4. Testing and Evaluation **22**

4.1 Function Test 22

4.2 Safety Test 25

4.3 System Evaluation and Improvements 26

5. Security Analysis **27**

5.1 Threat Identification 27

5.2 Cryptographic Evaluation 28

5.3 Secure System Design 29

6. Conclusion **30**

7. Reference **31**

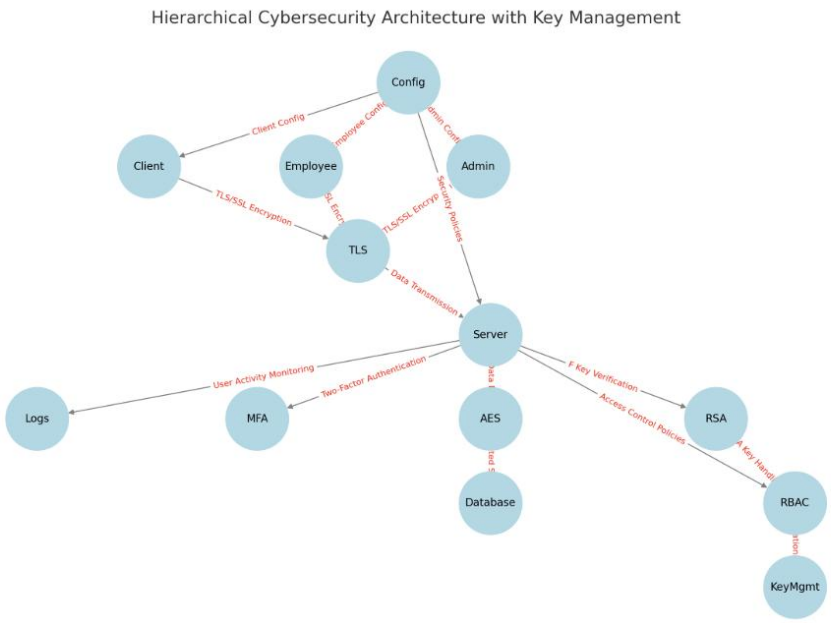
8. Appendix Credentials **32**

1. Introduction

To guard user information and financial transactions, online banking calls for robust security. While guaranteeing confidentiality, integrity, and authentication, this project plans a safe method to stop MITM attacks, account takeovers, and data breaches. For encryption, it uses AES-GCM; for secure communication, SSL/TLS; for key management, RSA; for authentication, MFA; and for access control RBAC. With careful role-based permissions, users may log in securely, manage accounts, and make encrypted transactions. Auditing and logging support system integrity maintenance, activity tracking, and security threat identification.

2. Design Justification

2.1 System Architecture



Overview

This system guarantees data security by means of a Client-Server (C/S) architecture including mechanisms of access control, key management, and safe communication.

Client Design

The client messages the server TLS/SSL encrypted and using multi-factor authentication (MFA) to guard against account compromise. By means of role-based access control (RBAC), different users—clients, bank employees, and administrators—only have access to approved functionality.

Servers Design

The server manages accounts, conducts transaction processing and user authentication. All critical information is encrypted using AES-256-GCM, and HKDF is utilized to generate keys therefore enhancing key security. The system maintains security monitoring and anomaly identification audit records.

Database Desgin

The database in MyBankProject is designed with a structured schema to ensure security, integrity, and scalability. It consists of nine key tables, each serving a specific function:

Table Name	Purpose	Key Features
Roles	Stores user roles and assigned permissions.	RBAC, JSON-based permissions.
Users	Manages user details, authentication, and role assignment.	Stores hashed passwords, MFA support, and public keys for encryption.
Accounts	Stores user accounts and balances.	Uses **AES-GCM** for encryption, enforces unique account numbers.
Transactions	Records all financial transactions.	Supports **encrypted details**, integrity hashing (**HMAC-SHA256**), and status tracking.
Audit Logs	Logs user actions for security and compliance.	Captures IP addresses, timestamps, and actions for monitoring.
Key Management	Handles cryptographic key storage and rotation.	Supports **RSA-2048**, key expiration, and key rotation tracking.
Offline Messages	Stores encrypted user messages.	Using **RSA public-key encryption** ensures secure message retrieval.

By means of MFA, encryption, and access control, this database design guarantees security and thereby lowers illegal access risks.

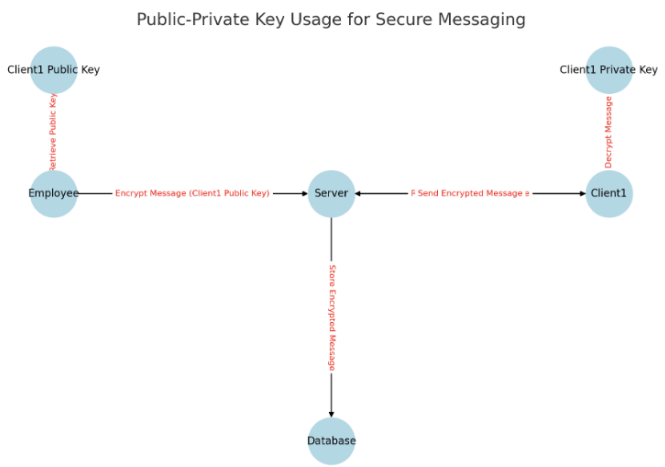
RBAC Design: Role-Based Access Control

Every system function in my RBAC (Role-Based Access Control) design is assigned rights. These rights are subsequently arranged and distributed to several roles, therefore guaranteeing obvious access separation depending on duties.

By allowing users to only access tasks pertinent to their position, this RBAC architecture improves security by lowering the possibility of illegal activity. It makes the system more controllable by eliminating pointless access to sensitive operations, therefore enhancing operational efficiency.

Role	Permissions
Client	read, execute, transfer_funds, withdraw_funds, deposit_funds, apply_for_loan, view_transactions, send_message, view_own_account, create_account
Employee	read, write, manage_transactions, approve_loans, deposit_funds, withdraw_funds, view_transactions, view_logs, view_all_accounts, manage_accounts, create_account
Admin	read, write, delete, manage_roles, view_logs, backup_system, update_system, create_account, approve_payments, create_user, delete_user, view_all_accounts, manage_accounts

RSA key pair Design in MyBank



RSA encryption allows MyBank to secure client-employee correspondence. Every user has a different key pair: the public key is kept on the server so others may encrypt communications; the private key stays with the user for decryption. This provides message integrity, ends-to-end security, and stops eavesdropping. Details are found on 3.1.2.

Key management Design

The MyBank banking system uses rigorous key management methods that guarantee user data integrity and confidentiality. Users' private keys are encrypted and safely kept through the key management module. Only admins can produce, rotate, delete, and check these encrypted keys to guarantee the general encryption system security.

Operation	Function	Security Mechanism	Use Case
Generate	Create new encryption keys	Uses CSPRNG for secure key generation and encrypted storage	New database encryption keys, user session keys
Rotate	Update existing keys to prevent exposure	Old keys revoked only after new keys take effect to prevent data loss	Periodic rotation of AES session keys, database keys
Revoke	Invalidate old keys to prevent misuse	Only administrators can revoke keys, ensuring they are no longer in use	Key compromise detected, system upgrade

2.2 Cryptographic Method Selection and Applicability

Section 2.2 presents a comparative table. It outlines each method’s advantages and limitations. Section 2.3 assesses encryption technologies based on security strength. It explains why they were chosen to ensure data confidentiality, secure communication, and access control within the system.

Table 1: Comparison of Encryption Methods

Encryption Method	Advantages	Disadvantages	Applicable Use Cases
AES-GCM Encryption	- Provides both confidentiality and integrity with built-in Tag - High efficiency suitable for high-concurrency environments	- Requires careful nonce management to avoid reuse risks	- Encrypting sensitive data in databases - Securing communication sessions - Protecting transaction records
HKDF Key Derivation	- Generates independent keys from a master key, reducing single-point failure - Enhances security by isolating encryption for different data fields	- Relies on the security of underlying HMAC - Correct salt usage is critical	- Generating unique encryption keys for different data types - Key diversification for user-specific encryption - Securing stored credentials
RSA 2048-bit Key Exchange	- Provides strong asymmetric encryption for secure key exchange - Widely standardized and supported	- Computationally intensive compared to symmetric methods - Slower large data volumes	- Securely exchanging session keys in key_management.py - Protecting short messages or critical authentication data
SHA256 + Salt (Password Hashing)	- Prevents direct exposure of passwords - Resists rainbow table attacks through salting	- Not specifically designed for passwords (compared to PBKDF2/Bcrypt)	- Secure password storage in authentication systems - Enhancing credential protection in database storage
SSL/TLS Communication	- Ensures end-to-end encryption and server authentication - Industry-standard for secure network communications	- Certificate management can be complex - Vulnerable if misconfigured	- Protecting data in transit between client and server - Preventing MITM attacks - Enforcing encrypted API communications

Session Key with AES-GCM	- Simplifies fast encryption/decryption of session data - Adds an extra layer of security beyond transport-level encryption	- Fixed session keys require periodic rotation to mitigate long-term risks	- Protecting short-lived session data - Strengthening secure communications beyond TLS - Encrypting cached sensitive data
Integrity Hash for Transactions	- Provides an extra integrity check at the database level to detect tampering - Complements AES-GCM's built-in integrity verification	- Increases implementation overhead - Requires secure hash computation and storage	- Ensuring transactional data integrity - Verifying logs and historical records - Detecting unauthorized database modifications
HMAC / Digital Signatures	- Enhances data authenticity and non-repudiation - Provides additional verification of critical data such as audit logs or sensitive transactions	- Adds extra computational load - Implementation complexity may increase if not integrated properly	- Securing transaction logs - Verifying message authenticity - Ensuring integrity of security-critical data

2.3 Explanation

The Reason for the Applicability of Encryption Methods in Table 2.2

2.3.1: Data Encryption Using AES-256-GCM

1. AES-256 boasts a 256-bit key, far stronger than 3DES (112-bit effective security). It can withstand current brute-force assaults.
2. Tag for integrity verification is part of built-in integrity check, AES-GCM. This eliminates the extra MAC required, hence optimizing encryption (Ferguson et al., 2003).

3.High Performance: For quick financial transactions, AES-256 performs admirably. Fast encryption and decryption are guaranteed by its optimization for contemporary CPUs.

2.3.2 Why Turn to HKDF for Key Derivation?

1. Using one key for several data types increases the chance of key misusing. HKDF generates special subkeys that protect other data should one key be hacked.
2. Improves Key Security: HKDF ensures long-term security by generating keys using HMAC, therefore masking the master key.
3. Adaptable for many purposes; fit for communication keys and storage encryption, so enhancing system security.

2.3.3: Key Exchange Using RSA-2048

1. RSA locks AES keys before they are sent. This keeps attackers away from intercepting them.
2. RSA-2048 tracks NIST guidelines in industry standards. Banking systems make great use of it for safe key management.
3. helps to lower RSA encrypts only temporary AES keys, therefore lowering the long-term key leakage risk.

2.3.4 Why Use TLS/SSL for Data Transmission?

1. TLS 1.3 guarantees that the client and server validate one another via certificates, therefore preventing data theft. This pauses MITM assaults.
2. Forward Secrecy (PFS) keeps prior communications safe even in cases if a private key leaks.
3. TLS 1.3 lowers handshake time than TLS 1.2, therefore accelerating transaction speed.

2.3.5 Password Hashing Using SHA-256 + Salt: Reasons

1. SHA-256 renders passwords irreversibly, so preventing password leaks. Attacks cannot recover passwords even in cases of database exposure.
2. Objectives resist Rainbow Table Attacks: Salting guarantees distinct hash results even if users have the same password.

3. Approved NIST-wise and extensively used in financial authentication systems is SHA-256.

2.3.6 Data Integrity Using HMAC-SHA256

1. HMAC with SHA-256 produces a safe message digest, therefore guaranteeing data integrity.
2. Attackers cannot replicate transactions without the proper HMAC key under high security.
3. Fast and Effective: HMAC is perfect for high-volume banking operations since it operates fast.

2.3.7 Why Encrypted with AES-GCM Session Keys?

1. Extra Security in Case of TLS Problems: Should TLS certificates be improperly configured; attackers could take over correspondence. Adding an extra encryption layer, AES-GCM protects data (Rescorla, 2001).
2. Perfect for real-time financial transactions since AES is faster than RSA.
3. AES-GCM uses a Tag to prevent session data tampering such that nobody may change it without being noticed.

2.3.8. Why might one use integrity hash for database security?

1. While Integrity Hash helps find illegal changes in storage, AES-GCM protects data during transmission.
2. Long-Term Security: Only upon decryption does AES-GCM verify integrity. Constant integrity check is offered by Integrity Hash.
3. Crucially for safe transactions: banking records must be tamper-proof. Integrity hash guarantees that every modification can be found.

3. Implementation Details

3.1 Main Functionality Implementation

3.1.1 Encryption/Decryption Module

This system guarantees integrity in storage and communication through data encryption using AES-GCM, thereby preserving confidentiality. Key exchange is done using RSA-2048, preventing symmetric keys from being intercepted during transmission. HKDF also generates separate keys for various users, reducing the single-point failure risk.

Implementation Details

- To guarantee data has not been altered, AES-GCM encrypts stored data and automatically creates a Tag for integrity verification.
- Dynamics generated keys to encrypt sensitive database fields including phone numbers and email addresses. HKDF generates these keys based on the username and field label, therefore guaranteeing that every user's data has a unique encryption key instead of depending just on a fixed key.
- Key transmission is done using RSA-OAEP, ensuring that AES symmetric keys are never leaked in plaintext over the network.
  HKDF Key Derivation
- For every user, HKDF creates unique field-specific keys. This improves security by preventing the same key from encrypting multiple data types.
-

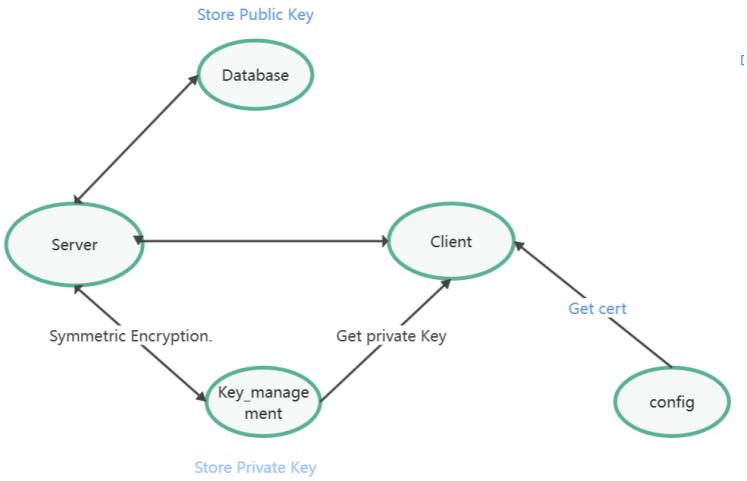
Data Encryption

The information from the server to the database is encrypted

Field	Storage Format	Encryption Method
Username	Plain text	No encryption
Password	Hashed (SHA-256 + Salt)	One-way hashing (not reversible)
Pay Password	Hashed (SHA-256 + Salt)	One-way hashing (not reversible)
Email	AES-GCM Encrypted (HKDF key)	Derived key using HKDF (per user, per field)
Phone Number	AES-GCM Encrypted (HKDF key)	Derived key using HKDF (per user, per field)

3.1.2 Key Management Strategy

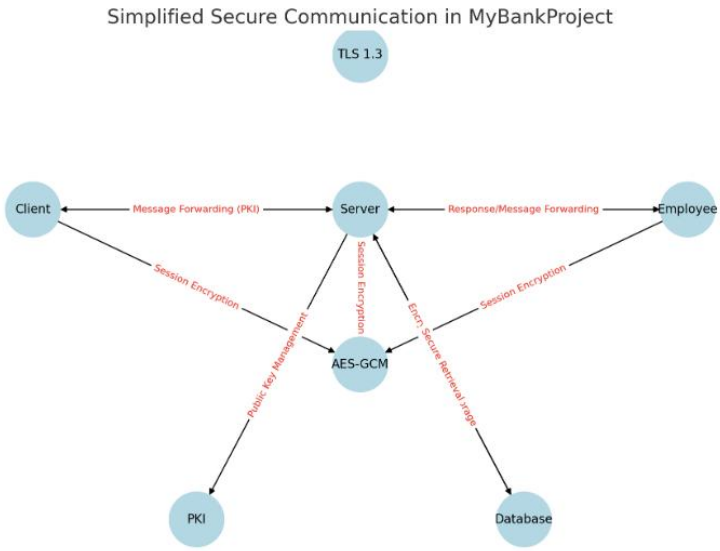
The system employs centralized key management, securely retrieving certificates and static keys from config.py, while storing private keys in Key Management instead of plaintext. Keys remain securely on the client and server, reducing the risk of unauthorized access and ensuring proper encryption key handling.



Implementation Details

- **Administrator Management:**
  Generating, distributing, rotating, and deleting encryption keys only by administrators with "manage_keys" helps to prevent both accidental and deliberate changes.
- **RSA 2048-bit for key exchange:**
  Between the client and the server, symmetric encryption keys are safely sent using RSA-2048 encryption, ensuring their impossibility of interception.
- **Important Rotation Mechanism:**
  By regularly updating AES encryption keys, the rotate_key() method limits their exposure and lowers the chance of compromise.
  Before being stored, new keys are encrypted using a Master Key, preventing direct access to unencrypted encryption keys.
- **Safe Key Protection:**
  Using AES-GCM, the encrypt_key_with_master() function encrypts every stored key, ensuring their protection from unauthorized access.
  Config.py centrally defines all encryption keys and security settings, maintaining a consistent security policy across the system.

3.1.3 Secure Communication Module



The MyBankProject system uses a public-key infrastructure (PKI) to provide safe message transfer among clients, staff, and the server. Whether a client or an employee, every user has a different public-private key pair that lets end-to-end encrypted communication possible.

Store encrypted messages and make sure only the intended receiver may decode them; the server serves as a middleman. TLS 1.3 encrypts all client-server contacts, therefore stopping illegal access to send data. Furthermore, an additional layer of AES-GCM session encryption is used to address possible flaws, therefore guaranteeing that the data stays encrypted even should TLS is hacked.

1. Public and Private Key Pairs

The system keeps at a minimum of two pairs of public-private keys: one for message authentication and another for user encryption. Retrieving their private keys from the Key Management system, users encrypt messages with the public key of the recipient therefore guaranteeing safe transmission. This ensures that the message may be decrypted and read only by the intended receiver together with the matching private key.

Roles	Public Key	Private Key
Client	client_public_key.pem	client_private_key.pem
Employee	employee_public_key.pem	employee_private_key.pem

```
sqlite> SELECT username, public_key FROM Users WHERE username = 'client1';
client1|-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEA11PYbFJ07H4Rww3gEm/C
0W9o25qJpiD/A5zYPdhtRwySNBRFSK2tEMm5uo0fWnXaIYItXVLDG/wdPWKkJrM5
fJsYrXBIoDb0ftXuci08JaEqSJbbHIZsr8EvFKgA9gzm9L/hVIvIcqFgMjssynr1
yma5XKW4FbK53FTDMAg7WmSwSf9e9hvxuCljS6MYBKuHOG9PrGRgvoVCFcSVjLnw
ylleeBLnCKJ4eVA7cu5XKbGS2z/KuuCGw5fiEKQ0iex1R4vNAfaXYELBFUDclRrE
CY4lGYDIelO3cD0A5lyLFwxr3U8D7zRNR6P9+d85fV9kh1RE3W0dyDQiR9QL28fJ
WQIDAQAB
-----END PUBLIC KEY-----
```

3-1. Public Key in database

Implementation Details

- **TLS/SSL Encryption:**
  With certificate and key file paths specified in config.py (CERTFILE, KEYFILE), the server authenticates using X.509 certificates.
  Support is mutual TLS (mTLS) authentication, ensuring client and server identity verification and preventing impersonation attacks.
- **Extra Security Using AES-GCM:**
  Every session creates a different AES-GCM encryption key, offering yet another degree of security beyond TLS.
  This extra encryption protects sensitive data even in cases of TLS compromise or misconfiguration.

3.1.4 Access Control and Verification

To guard user accounts and stop unwanted access, the system uses role-based access control (RBAC), multi-factor authentication (MFA), and password hashing. Table 2 lists twenty different permissions connected to server purposes. Table 3 shows the assigned rights for every role so that managers, staff members, and clients may only access the tools pertinent to their roles. Role authorization can be changed by administrators, so allowing flexible access control management.

Implementation Details

- **Password Security (SHA-256 + Salt):**

SHA-256 with a unique salt hashes user passwords, preventing attackers from guessing passwords from precomputed rainbow tables.

- **Multi-factor authentication (MFA):**
  The system creates a six-digit one-time password (OTP) when a user logs in (login_in_db()), requiring an extra verification step beyond a password.
- **RBAC, or role-based access control:**
  The database stores user roles and permissions, which are checked upon login. The system ensures that every user is limited to specific actions, preventing unauthorized access to critical capabilities.
  Config.py centrally manages all role definitions and access restrictions, ensuring security policy enforcement.

Table 3: Default Role-Based Access Control (RBAC) Permissions

Role (User Type)	Permissions Assigned
Client (client)	read, execute, transfer_funds, withdraw_funds, deposit_funds, view_own_account
Employee (employee)	view_all_accounts, manage_accounts, view_transactions, manage_transactions, approve_loans, create_account
Admin (admin)	view_all_accounts, manage_accounts, backup_system, update_system, create_user, delete_user, manage_roles, view_logs

3.1.5 Safety of Data Integrity

The system uses HMAC-SHA256 transaction hashing and AES-GCM authentication tags to enforce integrity verification and detect tampering, ensuring data accuracy.

Implementation Details

- **AES-GCM Integrity Check:**
  AES-GCM produces an authentication Tag when encrypting data, which is checked upon decryption to ensure the data has not been altered.
- **HMAC-SHA256 for use in transactions:**
  Every transaction run HMAC-SHA256, producing a secure hash stored in the Transactions table to prevent unauthorized changes.
- **Integrity Protection at Database Level:**
  Every transaction store an integrity hash, which allows managers to check that past information has not been changed during security audits.

3.2 Functionality and Encryption Methods

In the system, different functionalities use various encryption methods to ensure data security, authentication, and transaction integrity. The table presents the specific applications of AES-GCM, RSA-2048, HMAC-SHA256, TLS, OTP, digital signatures, and RBAC, meeting banking-grade security standards.

Table 2: Encryption Techniques Applied to System Functions

Function	Permission	Encryption Technique(s) Used
get_account_info_in_db()	read, view_own_account, view_all_accounts	AES-GCM (data encryption), TLS (secure transmission)
update_customer_info()	write, manage_accounts	AES-GCM (data encryption), Requires second password input
manage_user_roles()	delete, create_user, delete_user, manage_roles	AES-GCM (role storage), RBAC (role-based access control), Digital Signatures (Admin approval)
process_transfer()	execute, transfer_funds, manage_transactions	AES-GCM (transaction encryption), HMAC-SHA256 (integrity protection), Digital Signatures (Non-repudiation), Requires second password input
apply_for_loan()	approve_loans	AES-GCM (loan data encryption), RBAC (role validation), Digital Signatures (Loan approval authentication)
monitor_security()	view_logs	HMAC-SHA256 (log integrity), AES-GCM (log encryption)
system_maintenance()	backup_system, update_system	AES-GCM (backup encryption), RSA-2048 (key management for system updates), Digital Signatures (System update verification), Requires second password input
process_withdrawal()	withdraw_funds	AES-GCM (transaction data encryption), HMAC-SHA256 (integrity verification), Digital Signatures (Withdrawal confirmation), Requires second password input

process_deposit()	deposit_funds	AES-GCM (deposit data encryption), HMAC-SHA256 (transaction integrity), Digital Signatures (Deposit confirmation)
monitor_transactions()	view_transactions	HMAC-SHA256 (transaction log integrity), RBAC (access control), Digital Signatures (Audit log authentication)
modify_account_info_in_db()	manage_accounts	AES-GCM (account encryption), TLS (secure access), Digital Signatures (Critical changes verification), Requires second password input
create_account_in_db()	create_account	AES-GCM (secure account storage), RSA-2048 (key generation for new accounts), Digital Signatures (Account creation verification)

3.3 User Role Functionality Implementation

The banking system employs a permission management framework based on three primary roles: Client, Bank Employee, and System Administrator, with all permissions stored in the Roles table to ensure clear operational boundaries for different users.

3.3.1 Client Functionality

Account Registration:
create_account_in_db() validates username format, password strength, email, and phone number. Passwords and payment PINs are encrypted using SHA-256 with salt before storage. Users with create_account permission automatically generate a bank account (stored in the Accounts table).
Account Login (with OTP Two-Factor Authentication):
login_in_db() verifies username, password, and OTP codes. Upon successful login, the system records the user's role_id and assigns corresponding permissions.
Retrieve Account Information:

get_account_info_in_db() queries the Accounts table for account details, the Transactions table for transaction history, and the Loans table for loan status. Sensitive data is decrypted before being returned to the user.

```
[DEBUG] Decryption Successful: {"action": "login", "username": "client1", "password": "******", "otp": ""}
{'action': 'login', 'username': 'client1', 'password': '******'}
{'action': 'login', 'username': 'client1', 'password': '******'}
```

3-2 Implementation details

Fund Transfer:

process_transfer() checks transfer_funds permission and payment PIN, verifies account balance, deducts funds, and logs the transaction in Transactions.

Bill Payment:

pay_bill() verifies pay_bill permission and payment PIN, completes the payment, and records it in Transactions.

Loan Application:

apply_for_loan() checks apply_for_loan permission and stores the application in the Loans table with a pending status.

Secure Messaging:

send_message() encrypts messages using the recipient's RSA public key and stores them in the Offline_Messages table to ensure secure communication between clients and bank representatives.

```
----- Employee Login -----
Enter username: employee1
Enter password: 123456
OTP has been sent. Check the file 'otp_employee1.txt'.
Enter OTP: 476198
✅ Login successful! Welcome, employee1

==== Employee Panel (Logged in as: employee1) ====

📩 Unread Messages:
[WARNING] Retrieved plaintext private key for employee1. Consider encrypting it.
📨 From 1: hello
📨 From 1: hello
```

3-3 Implementation details

3.3.2 Bank Employee Functionality

View Customer Information:

View_customer_info_in_db() checks the requester's employment status, gathers data from the Users and Accounts tables, then decodes private data employing HKDF-derived encryption keys. Based on the username and field label—say, "email," "phone"—the system generates unique decryption keys per user and field using HKDF.

```
Database Query Result:  (7, 'client6', '{"nonce": "lFdhxNYcpDqzn
[DEBUG] Derived Email Key: 976b53e62c2a83c3667ab38b5347097148fe4
[DEBUG] Derived Phone Key: 7f4b01389b7f42781edbd7b40354f7ab15bab
[DEBUG] Decryption Inputs - Nonce: 945761c4d61ca43ab39df2ebdafa1
[DEBUG] AES Decrypt - Nonce: 945761c4d61ca43ab39df2ebdafa1185, C
[DEBUG] Decryption Successful: client6@qq.com
[DEBUG] Decryption Inputs - Nonce: b7e4ae3829b5bd444f57419ffcf66
[DEBUG] AES Decrypt - Nonce: b7e4ae3829b5bd444f57419ffcf66d29, C
[DEBUG] Decryption Successful: 0771234
{'action': 'login', 'username': 'client1', 'password': '******'}
```

3-4 Implementation details

Process Deposits:

process_deposit() checks deposit_funds permission, updates account balances in Accounts, and logs transactions in Transactions.

Process Withdrawals:

process_withdrawal() verifies withdraw_funds permission, confirms sufficient balance, deducts funds, and records the transaction.

Loan Approval:

approve_loans() modifies the status of entries in the Loans table to approved or denied.

Transaction Monitoring:

monitor_transactions() checks view_transactions permission and retrieves recent records from the Transactions table.

3.3.3 System Administrator Security Management

User Role Management:

manage_user_roles() enables administrators to create, modify, delete, and view roles and permissions in the Roles table.

Key Management:

manage_keys() allows generation, rotation (rotate_key()), or revocation of encryption keys, stored in the Key_Management table.

```
==== Admin Panel (Logged in as: admin1) ====
3 - Manage User Roles
4 - Monitor Security
5 - Manage Keys
6 - System Maintenance
8 - Logout
Enter your choice: 5
Re-enter admin password for verification: 666666
Enter key management operation (generate/rotate/revoke/view): generate
Enter key type (AES, RSA, etc.): AES
Enter key usage (general, session, encryption): general
Manage Keys response: {'status': 'success', 'message': 'Key generated and securely stored'}
```

3-5 Implementation details

System Maintenance:

system_maintenance() executes system updates (e.g., apt-get update), applies security patches (e.g., OpenSSL updates), and performs database backups (e.g., sqlite3 .backup).

Security Event Monitoring:

monitor_security() queries the Audit_Logs table to extract security-related events such as intrusion attempts, failed logins, and audit trails for review.

```
CONNECT: ('127.0.0.1', 63643) connected
REQUEST: ('127.0.0.1', 63643) request: {'action': 'login', 'username': 'client1', '******'
ERROR: ('127.0.0.1', 63643) handling error: [SSL: TLSV1_ALERT_PROTOCOL_VERSION] tlsv1 aler
DISCONNECT: ('127.0.0.1', 63643) disconnected
CONNECT: ('127.0.0.1', 63663) connected
REQUEST: ('127.0.0.1', 63663) request: {'action': 'login', 'username': 'client1', '******'
ERROR: ('127.0.0.1', 63663) handling error: [SSL: TLSV1_ALERT_PROTOCOL_VERSION] tlsv1 aler
DISCONNECT: ('127.0.0.1', 63663) disconnected
```

3-6 Log Details

4. Testing and Evaluation

4.1 Function test

This stage confirms the fundamental features of the system among managers, bank staff, and clients. It guarantees expected performance of account management, transactions, authentication, permissions, logging systems, and administrative controls.

The appendix contains references including test results and screenshots.

Test ID	Feature	User Role	Input Data	Expected Output	Actual Result	Conclusion
FT-01	Account Registration	Client	Valid username, password, email, phone number	Account successfully created	Account successfully created	Passed
FT-02	Account Registration	Client	Duplicate username	Registration fails with "Username already exists" message	Registration fails with correct error message	Passed
FT-03	Account Registration	Employee	Valid username, password, email, phone, valid employee registration code	Account successfully created	Account successfully created	Passed
FT-04	Account Registration	Employee	Valid username, password, email, phone, missing or invalid registration code	Registration fails	Registration fails with correct error message	Passed
FT-05	Account Registration	Admin	Valid username, password, email, phone, valid admin registration code	Account successfully created	Account successfully created	Passed
FT-06	Account Registration	Admin	Valid username, password, email, phone, missing or invalid	Registration fails	Registration fails with correct error	Passed

			registration code		message	
FT-07	Login	Client	Valid username + password + OTP	Successful login	Successful login	Passed
FT-08	Login	Client	Incorrect OTP	Login fails	Login fails with correct error message	Passed
FT-09	Money Transfer	Client	Valid source and destination accounts with sufficient balance	Transfer completes successfully	Transfer completed ; balance updated	Passed
FT-10	Money Transfer	Client	Source accounts have insufficient balance	Transfer fails with "Insufficient balance" message	Transfer failed with correct error message	Passed
FT-11	Bill Payment	Client	Valid account number and sufficient balance	Payment completes successfully	Payment completed ; balance updated	Passed
FT-12	Loan Applicati on	Client	Valid requests with required fields	Loan application submitted successfully	Loan applicatio n submitted and recorded	Passed
FT-13	Loan Approval	Employ ee	Approving a pending loan	Loan status changes to "Approved"	Loan status updated correctly	Passed
FT-14	Viewing Transacti ons	Client	Valid account owner	Transaction s are retrieved successfully	Transactio ns retrieved correctly	Passed
FT-15	Viewing Transacti ons	Employ ee	Employee viewing all transactions	Access granted	Transactio ns retrieved correctly	Passed
FT-16	Viewing Transacti ons	Unauth orized User	Unauthorized user accessing another account	Access denied	Access denied correctly	Passed
FT-17	Role-	Employ	Employee	Access	Access	Passed

	Based Access Control (RBAC)	ee	attempting admin-level actions	denied	denied correctly	
FT-18	User Role Manage ment	Admin	Creating, updating, deleting roles	Role changes applied successfully	Role changes applied correctly	Pass

Results Analysis

1. Only users with valid registration codes can register as employees or admins, preventing unauthorized registrations.
2. RBAC enforcement stops illegal behavior rather effectively.
3. Robust systems for transaction validation and authentication help to reduce security vulnerabilities.
4. Administrative controls—role management and security monitoring among other things—function as expected.

4.2 Safety test

Test ID	Attack Type	Test Method	Test Result	Mitigation Recommendations
ST-01	SQL Injection	Input username = "admin' OR '1'='1" into login form	Login fails; no data leaked	Parameterized queries prevent SQL injection
ST-02	Unauthorized Employee Registration	Attempt to register as an employee without a valid registration code	Registration fails	Employee registration requires a valid code
ST-03	Unauthorized Admin Registration	Attempt to register as an admin without a valid registration code	Registration fails	Admin registration requires a valid code
ST-04	Horizontal Privilege Escalation	Attempting to access another client's transaction history	Access denied	RBAC enforcement is effective
ST-05	Vertical Privilege Escalation	Employee attempting to modify admin settings	Access denied	Role-based access controls are properly implemented
ST-06	Session Hijacking	Copy and use another client's JWT Token	Requires OTP re-authentication	Implement shorter session expiration time
ST-07	Replay Attack	Resend previously signed transfer request	Transfer request fails	Implement unique nonce for transaction requests
ST-08	Log Data Exposure	Read server.log	Logs are accessible	Encrypt logs using AES before storage
ST-09	Brute Force Attack	Repeated login attempts with	Account locked after multiple	Rate limiting and CAPTCHA

		incorrect passwords	failures	recommended for enhancement
ST-10	Unauthorized Role Assignment	Attempt to assign admin rights to an employee account via direct database access	System prevents unauthorized role modification	Secure access controls in database management verified
ST-11	Unauthorized Loan Approval	Attempt to approve a loan for a non-existent user	Approval fails	Input validation and database integrity maintained
ST-12	Credential Exposure in Logs	Analyze log files for stored credentials	No passwords found in logs	Sensitive data logging is restricted, no credentials stored in logs

4.3 System Evaluation and Improvements

By means of parameterized searches, RBAC implementation, and access control methods, the security testing results show that the system efficiently stops SQL injection, unauthorized role registration, and privilege escalation. Still, session hijacking hazards exist since session expiration calls for optimization. A problem is replaying attacks, which call for nonce-based validation of transaction integrity. Furthermore, log data exposure presents security concerns that call for AES encryption for logs. Though CAPTCHA deployment improves login security, brute-force defense is only partially successful. By means of strengthening these areas, system resilience and adherence to cybersecurity best practices would be much enhanced.

5. Security Analysis

5.1 Threat Identification

The system's security evaluation points out main hazards connected to administrators, bank staff, and consumers. Based on the STRIDE threat model, the following table lists security risks together with assessments of current security policies, found problems, and suggested enhancements.

Threat Category	Risk Description	Existing Security Measures	Identified Issues	Proposed Improvements
Spoofing (Identity Impersonation)	Unauthorized users may impersonate legitimate users to gain access to the system.	Two-Factor Authentication (OTP) is enforced during login to strengthen authentication.	No significant issues identified.	No changes required.
Tampering (Data Integrity Risks)	Unauthorized modification of transaction data could compromise financial integrity.	AES-GCM encryption is utilized to ensure data confidentiality and integrity. Digital signatures provide additional protection.	No significant issues identified.	No changes required.
Repudiation (Non-Repudiation Risks)	Users may deny performing specific actions within the system.	Encrypt logs using AES before storage.	No significant issues identified.	No changes required.
Information Disclosure (Data Leakage)	Sensitive user data (such as emails and phone	Email addresses and phone numbers are encrypted prior	STATIC_KEY is hardcoded in the codebase, creating a	Store encryption keys securely using a Hardware

	numbers) may be accessed by unauthorized parties.	to storage. A key rotation mechanism is in place to mitigate long-term exposure.	security risk if compromised.	Security Module (HSM) or a cloud-based Key Management System (KMS).
Denial of Service (DoS)	Attackers may generate excessive authentication requests, leading to system disruption.	No rate limiting mechanism has been implemented for login attempts.	The absence of rate limiting exposes the system to brute-force attacks.	Introduce login failure rate limiting (e.g., fail2ban, exponential backoff). Require CAPTCHA or secondary authentication after multiple failed login attempts.
Elevation of Privilege	Unauthorized users may exploit vulnerabilities to gain elevated system privileges.	Role-Based Access Control (RBAC) restricts access based on predefined roles.	No significant issues identified.	No changes required.
Repudiation (Non-Repudiation Risks)	Users may deny performing specific actions within the system.	The log_audit() function records user actions, enabling traceability and accountability.	Logs are stored in plain text, posing a risk of exposure to unauthorized access.	Encrypt logs using AES before storage. Implement WORM (Write Once Read Many) storage to prevent log tampering.

5.2 Cryptographic Evaluation

The system utilizes SHA-256, AES-GCM, RSA, and HKDF. The following table evaluates their implementation:

Algorithm	Usage	Strengths	Issues	Proposed Improvements
SHA-256 + salt	Password hashing	Strong collision resistance	Lacks iteration mechanism, susceptible to brute-force attacks	Replace with Argon2 or bcrypt
AES-GCM (256-bit key)	Data encryption	Provides confidentia lity and integrity	Nonce reuse risk	Ensure unique nonce generation
RSA (2048-bit key)	Message encryption	Secure asymmetric encryption	Key length is too short (should be at least 4096 bits)	Upgrade to 4096-bit RSA for enhanced security
HKDF (SHA-256)	Key derivation	Strong key derivation mechanism	Hardcoded master key storage	Use KMS/HSM for secure key management

Key Cryptographic Issues

Stronger resilience against brute-force attacks calls for Argon2 or bcrypt, not SHA-256 for password hash.
To guarantee long-term security, RSA keys should be minimum forty-six thousand bits. One main weakness is STATIC_ KEY hardcoding; so, one should make use of safe storage options including HSM/KMS.

5.3 Secure System Design Evaluation

The following table evaluates how well the system adheres to secure system design principles:

Design Principle	Evaluation	Identified Issues	Proposed Improvements
Least Privilege	RBAC implemented	No issues	No changes needed
Defense in Depth	Multi-layer encryption and	No HMAC for request verification	Implement TOTP and HMAC for API

	authentication		request validation
Separation of Duties	Role-based permission enforcement	Critical operations can be executed by a single individual	Implement dual approval for large transactions
Fail-Safe Defaults	Default access is denied	Some API permissions are too permissive	Ensure all API endpoints enforce strict authentication
Economy of Mechanism	Uses standard cryptographic libraries	Static keys stored insecurely	Use HSM/KMS for key storage
Complete Mediation	Some API requests validate permissions	Lack of session timeout enforcement	Implement session expiration & mandatory re-authentication
Open Design	Uses standard cryptographic algorithms	No issues	No changes nee

Analysis

The system has to encrypt important log data, safely handle encryption keys with HSM/KMS, and apply rate-limiting systems to stop brute-force attacks in order to lower these hazards. By tackling these issues, the system will improve its resistance to security risks, therefore guaranteeing compliance with ISO 27001 and the NIST Cybersecurity Framework (CSF).

6. Conclusion

This paper evaluated security threats in the financial system using the STRIDE framework comprising 2FA, AES-GCM encryption, digital signatures, and RBAC. Still present, meanwhile, are flaws including absence of rate limitation, hardcoded encryption keys, and unencrypted audit logs. Dealing with these calls for encrypted records, safe key management (HSM/KMS), and brute-force defense. In access control and authentication, juggling security with usability still presents a difficult problem. Future developments should concentrate on automating important rotation, improving session management, and extending MFA outside OTP. By strengthening these policies, system resilience will increase and compliance with ISO 27001 and NIST CSF will be assured.

7. References:

1. Bonneau, J., Herley, C., Van Oorschot, P.C., and Stajano, F. (2012) 'The Quest to Replace Passwords: A Framework for Comparative Evaluation of Web Authentication Schemes', IEEE Symposium on Security and Privacy, pp. 553–567.

2. Dierks, T. and Rescorla, E. (2008) The Transport Layer Security (TLS) Protocol Version 1.2. RFC 5246.

3. Ferguson, N., Schneier, B., and Kohno, T. (2003) Cryptography Engineering: Design Principles and Practical Applications. Wiley Publishing.

4. Krawczyk, H. (2010) 'HKDF: Key Derivation Using HMAC', RFC 5869.

5. National Institute of Standards and Technology (2002) FIPS PUB 180-2: Secure Hash Standard (SHS).

6. Rescorla, E. (2001) SSL and TLS: Designing and Building Secure Systems. Addison-Wesley.

7. Stallings, W. (2016) Cryptography and Network Security: Principles and Practice. 7th edn. Pearson.

8. Schneier, B. (1996) Applied Cryptography: Protocols, Algorithms, and Source Code in C. 2nd edn. Wiley.

9. Viega, J. and McGraw, G. (2001) Building Secure Software: How to Avoid Security Problems the Right Way. Addison-Wesley.

10. Shostack, A. (2014) Threat Modeling: Designing for Security. Wiley.

11. Open Web Application Security Project (OWASP) (2021) OWASP Top Ten Security Risks. Available at: https://owasp.org/www-project-top-ten/ (Accessed: Date).

12. National Institute of Standards and Technology (2017) Special Publication 800-63B: Digital Identity Guidelines – Authentication and Lifecycle Management.

8. Appendix: Test Credentials

Basic User Credentials

User Role	Username	Password	Secondary Password
Client	client1	123456	666666
Employee	employee1	123456	666666
Administrator	admin1	123456	666666

Employee and Admin Registration Codes

Role	**Registration Code**
Employee	EMP2023
Administrator	ADM2023

The images below are arranged in test order, and the test functions are shown in the figure

8.1 The implementation of the main user functions is as follows:

Registration requirements

```
# Between **3 to 20 digits** long.
def validate_username(username):
    return re.match( pattern: r"^\w{3,20}$", username) is not None

#follow the format: **local_part@domain.extension**
def validate_email(email):
    return re.match( pattern: r"^[\w\.-]+@[\w\.-]+\.\w+$", email) is not None

# Between **6 to 15 digits** long.
def validate_phone(phone):
    return re.match( pattern: r"^\d{6,15}$", phone) is not None
```

Main client functions

```
----- Register New Account -----
Enter username: client6
Enter password: 123456
Enter email: client6@qq.com
Enter phone number: 0771234
Enter payment password: 666666
Registration response: {'status': 'success', 'message': 'Account created successfully', 'user_id': 7}
```

8-1 Account creation success

```
Logged in as: client1
3 - Get Account Information
4 - Transfer Funds
5 - Pay Bill
6 - Update Personal Information
7 - Apply for Loan
8 - Send Message to Bank Representative
9 - Logout
Enter your choice: 8
----- Send Message -----
Enter recipient username: employee1
Enter message: 123456
📨 Message response: Encrypted message sent
✅ No new messages.
Logged in as: client1
3 - Get Account Information
4 - Transfer Funds
5 - Pay Bill
6 - Update Personal Information
7 - Apply for Loan
8 - Send Message to Bank Representative
9 - Logout
Enter your choice: 8
----- Send Message -----
Enter recipient username: client1
Enter message: 1244
📨 Message response: Encrypted message sent
```

```
Enter your choice: 3
----- Get Account Information -----
Account Information: {'status': 'success', 'accounts': [], 'transactions': [],
Logged in as: client1
3 - Get Account Information
```

```
Enter your choice: 4
----- Transfer Funds -----
Enter source account number: 077123
Enter destination account number: 077123
Enter transfer amount: 30
Enter currency (default USD):
Enter payment password for verification: 666666
Transfer response: Transfer initiated successfully
Logged in as: client1
```

```
Enter your choice: 5
----- Pay Bill -----
Enter account number for payment: 07712
Enter biller name: biller1
Enter payment amount: 30
Enter payment password for verification: 666666
Bill payment response: Account not found
Logged in as: client1
```

```
Enter your choice: 6
----- Update Personal Information -----
Enter new address: CV1
Enter new phone number: 077123
Update response: Unknown action
Logged in as: client1
```

8.2 The implementation of the main employee functions is as follows:

The user information is found and decrypted successfully. The key of client1 has been changed for test, please find another user (client2) for test or a newly registered account.

```
==== Employee Panel (Logged in as: employee1) ====
✅ No new messages.
3 - View Customer Information
4 - Process Deposit
5 - Process Withdrawal
6 - Process Transfer
7 - Update Customer Information
8 - Monitor Transactions
9 - Send Message
0 - Logout
Enter your choice: 3
----- View Customer Information -----
Enter customer username: client6
Customer Information:
Username: client6
Email: client6@qq.com
Phone: 0771234
```

```
Database Query Result:  (7, 'client6', '{"nonce": "lFdhxNYcpDqzn
[DEBUG] Derived Email Key: 976b53e62c2a83c3667ab38b5347097148fe4
[DEBUG] Derived Phone Key: 7f4b01389b7f42781edbd7b40354f7ab15bab
[DEBUG] Decryption Inputs - Nonce: 945761c4d61ca43ab39df2ebdafa1
[DEBUG] AES Decrypt - Nonce: 945761c4d61ca43ab39df2ebdafa1185, C
[DEBUG] Decryption Successful: client6@qq.com
[DEBUG] Decryption Inputs - Nonce: b7e4ae3829b5bd444f57419ffcf66
[DEBUG] AES Decrypt - Nonce: b7e4ae3829b5bd444f57419ffcf66d29, C
[DEBUG] Decryption Successful: 0771234
{'action': 'login', 'username': 'client1', 'password': '******'}
```

8-2 The database information decryption log seen on the server

```
----- Employee Login -----
Enter username: employee1
Enter password: 123456
OTP has been sent. Check the file 'otp_employee1.txt'.
Enter OTP: 650667
✅ Login successful!
✅ No new messages.

==== Employee Panel (Logged in as: employee1) ====
3 - View Customer Information
4 - Process Deposit
5 - Process Withdrawal
6 - Process Transfer
7 - Update Customer Information
8 - Monitor Transactions
9 - Send Message
0 - Logout
Enter your choice: 9
----- Send Message -----
Enter recipient username: client1
Enter message: hello
📨 Message response: Encrypted message sent
✅ No new messages.
```

The functions that client1 sends to employee1 are securely transmitted by using asymmetric encryption.

```
----- Employee Login -----
Enter username: employee1
Enter password: 123456
OTP has been sent. Check the file 'otp_employee1.txt'.
Enter OTP: 476198
✅ Login successful! Welcome, employee1

==== Employee Panel (Logged in as: employee1) ====

📩 Unread Messages:
[WARNING] Retrieved plaintext private key for employee1. Consider encrypting it.
📨 From 1: hello
📨 From 1: hello
```

8-3 client1 sends to employee1

Also using **Test_communication** to simulate a secure conversation between client1 and employee1.

This script:

1. **Encrypts a message** from client1 using employee1's public key.
2. **Decrypts the message** using employee1's private key.
3. **Encrypts a response** from employee1 using client1's public key.
4. **Decrypts the response** using client1's private key.
5. **Prints the input, encrypted, and decrypted messages for both users.**

```
"D:\Study\Applied Cryptography\cwk\MybankProject\.venv\Scripts\python.e
Enter the message to encrypt: hello

◆ Testing encryption and decryption for client1 ◆
Input message: hello
Encrypted message (hex) with client1's public key: 45ac5f4e0a951541
Decrypted message with client1's private key: hello

◆ Testing encryption and decryption for employee1 ◆
Input message: hello
Encrypted message (hex) with employee1's public key: 168ee5458a738b
Decrypted message with employee1's private key: hello
```

8-2 **Test_communication.py** to simulate a secure conversation

8.3 The implementation of the main admin functions is as follows：

```
==== Admin Panel (Logged in as: admin1) ====
3 - Manage User Roles
4 - Monitor Security
5 - Manage Keys
6 - System Maintenance
8 - Logout
Enter your choice: 3
Re-enter admin password for verification: 666666
Enter role management operation (create/update/delete/view): view

==== Existing Roles ====
ID: 13 | Name: client | Permissions: ['read', 'execute', 'transfer_funds', 'withdraw_funds', 'deposit_funds', 'apply_fo
ID: 14 | Name: employee | Permissions: ['read', 'write', 'manage_transactions', 'approve_loans', 'deposit_funds', 'with
ID: 15 | Name: admin | Permissions: ['read', 'write', 'delete', 'manage_roles', 'view_logs', 'backup_system', 'update_s
ID: 20 | Name: auditor | Permissions: ['read', 'view_logs', 'view_transactions']
```

```
==== Admin Panel (Logged in as: admin1) ====
3 - Manage User Roles
4 - Monitor Security
5 - Manage Keys
6 - System Maintenance
8 - Logout
Enter your choice: 5
Re-enter admin password for verification: 666666
Enter key management operation (generate/rotate/revoke/view): generate
Enter key type (AES, RSA, etc.): AES
Enter key usage (general, session, encryption): general
Manage Keys response: {'status': 'success', 'message': 'Key generated and securely stored'}
```

```
==== Admin Panel (Logged in as: admin1) ====
3 - Manage User Roles
4 - Monitor Security
5 - Manage Keys
6 - System Maintenance
8 - Logout
Enter your choice: 6
Re-enter admin password for verification: 666666
Enter system maintenance operation (update/patch/backup): update
Enter update description: This update enhances system security, performance
System Maintenance response: {'status': 'error', 'message': "System mainter
```

8.4 Security testing

Error Input Detection

```
Enter your choice: 2
----- Client Login -----
Enter username: client1
Enter password: 12356
❌ Login failed: Incorrect password
```

```
Enter your choice: 2
----- Client Login -----
Enter username: client1
Enter password: 123456
OTP has been sent. Check the file 'otp_client1.txt'.
Enter OTP: 122222
❌ Login failed: Incorrect OTP.
```

```
==== Admin Client ====
1 - Register
2 - Login
9 - Exit
Enter your choice: 2
----- Admin Login -----
Enter admin username: client1
Enter password: 123456
OTP has been sent. Check the file 'otp_client1.txt'.
Enter OTP: 720634
❌ Access denied: Only admins can log in here.
```

```
----- Employee Login -----
Enter username: client1
Enter password: 123456
OTP has been sent. Check the file 'otp_client1.txt'.
Enter OTP: 786892
❌ Access denied: Only employees can log in here.
🔒 Exiting employee panel due to failed login.
```

```
Enter your choice: 2
----- Client Login -----
Enter username: admin1
Enter password: 123456
OTP has been sent. Check the file 'otp_admin1.txt'.
Enter OTP: 971437
❌ Access denied: Only clients can log in here.
🔒 Login failed. Please try again.
```

```
==== Admin Client ====
1 - Register
2 - Login
9 - Exit
Enter your choice: 2
----- Admin Login -----
Enter admin username: client1
Enter password: 123456
OTP has been sent. Check the file 'otp_client1.txt'.
Enter OTP: 592179
❌ Access denied: Only admins can log in here.
```

```
==== Employee Panel (Logged in as: employee1) ====
✅ No new messages.
3 - View Customer Information
4 - Process Deposit
5 - Process Withdrawal
6 - Process Transfer
7 - Update Customer Information
8 - Monitor Transactions
9 - Send Message
0 - Logout
Enter your choice: 3
----- View Customer Information -----
Enter customer username: client5
Failed to retrieve customer information: Customer not found
```

```
✅ No new messages.
----- Transfer Funds -----
Enter source account number: 12
Enter destination account number: 12
Enter transfer amount: 12
Enter currency (default USD):
Enter payment password for verification: 777777
Transfer response: Payment password verification failed
Logged in as: client1
```

8-4 Payment code error alert

```
==== Employee Panel (Logged in as: employee1) ====
✅ No new messages.
3 - View Customer Information
4 - Process Deposit
5 - Process Withdrawal
6 - Process Transfer
7 - Update Customer Information
8 - Monitor Transactions
9 - Send Message
0 - Logout
Enter your choice: 3
----- View Customer Information -----
Enter customer username: client1
Failed to retrieve customer information: Permission denied: employee does not have access to view customer information
```

8-4 Test for ‘No permission'.

```
Enter your choice: 9
----- Send Message -----
Enter recipient username: client5
Enter message: Hello
📨 Message response: Recipient user does not exist
✅ No new messages.
```

8-4 Test exists for no receiving user

```
Enter your choice: 9
----- Send Message -----
Enter recipient username: client2
Enter message: hello
📨 Message response: Recipient does not have a public key registered
✅ No new messages.
```

8-4 No public key test